

LLDD BE - Job 6 ExportImpactStoreToFS

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	32 ชั่วโมง = implementation 24 + unit test 8 (30%)
Owner	Aphiwit <Bank> Khammoon
Target repository	SBP/srm-sps-spsap-sop-sgi-batch (NestJS 11 + TypeORM · schema sps_store · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน src/main.ts แล้วรับ argument ผ่าน JOB_NAME/INPUT (local) หรือ argv[3]/argv[2] (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ @Cron · ดู srm sps spsap sop sgi batch
Objective	ซิงค์สถานะ + ส่งค่าชดเชยไป STA: รัน 10 mutation ตามลำดับบนตารางสถานะ ตรวจสอบความครบของคะแนน QSSI 6 หมวด สร้างชุดสถานะที่ส่งออกได้ แล้ว publish message ไป RabbitMQ ให้ระบบ Statement (STA) รับต่อ (มติ 2026-08-24 — เลิกเขียนไฟล์ FRBC0001 + SFTP) · เนื้อข้อมูลยังเป็นสัญญาเดิม 14 ฟิลด์ แต่เป็น JSON UTF-8 ไม่ใช่ text windows-874 · ใช้ transactional outbox แบบเดียวกับ Job 4 — DB transaction ครบ sync + outbox แล้วค่อย publish นอก transaction

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope {success, data} · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (x-api-key / x-user-*) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-2-ImportImpactStore.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

- Main class/script: fgi.main.ExportImpactStoreToFS / FGI_ExportImpactStoreToSTA.sh
- Phase: D
- Output: RabbitMQ message sgi_impact_store (exchange sgi.interface)
- Estimate: 24 ชั่วโมง
- Argument รับผ่าน INPUT (JSON) — local ใช้ env JOB_NAME/INPUT · AWS Batch ใช้ argv[3]/argv[2] · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอบริการ (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี @Cron) · ทุก job ถูกบันทึก integration_log โดย main.ts อัปเดตโน้ต + structured log BATCH_START/BATCH_END พร้อม runId

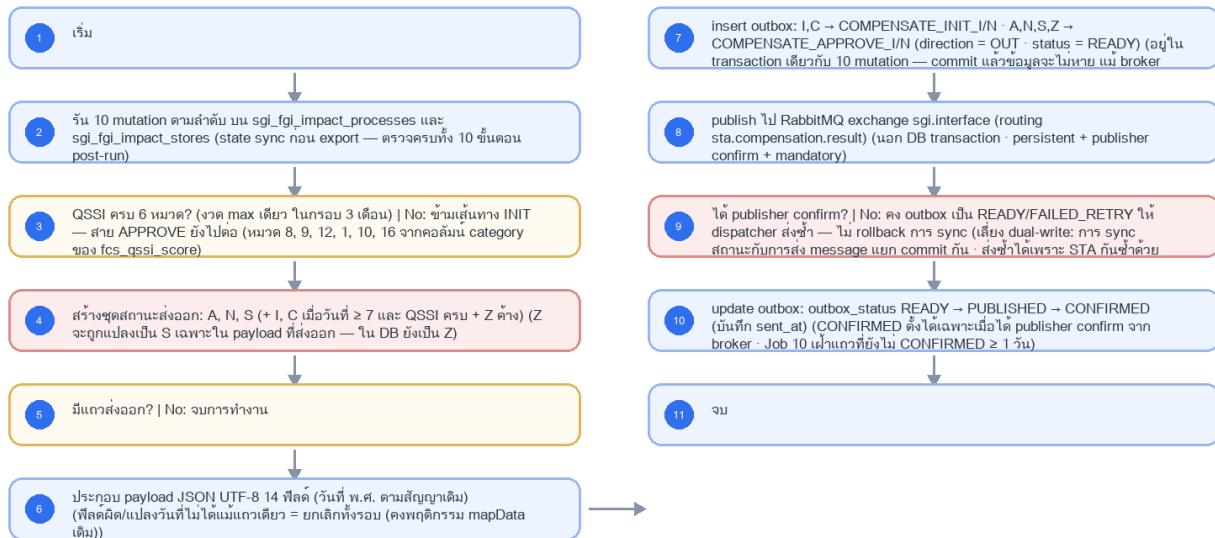
3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

4.1 Implementation Flow (ลำดับขั้นตอนการทำงาน)

LLDD BE - Job 6 ExportImpactStoreToFS

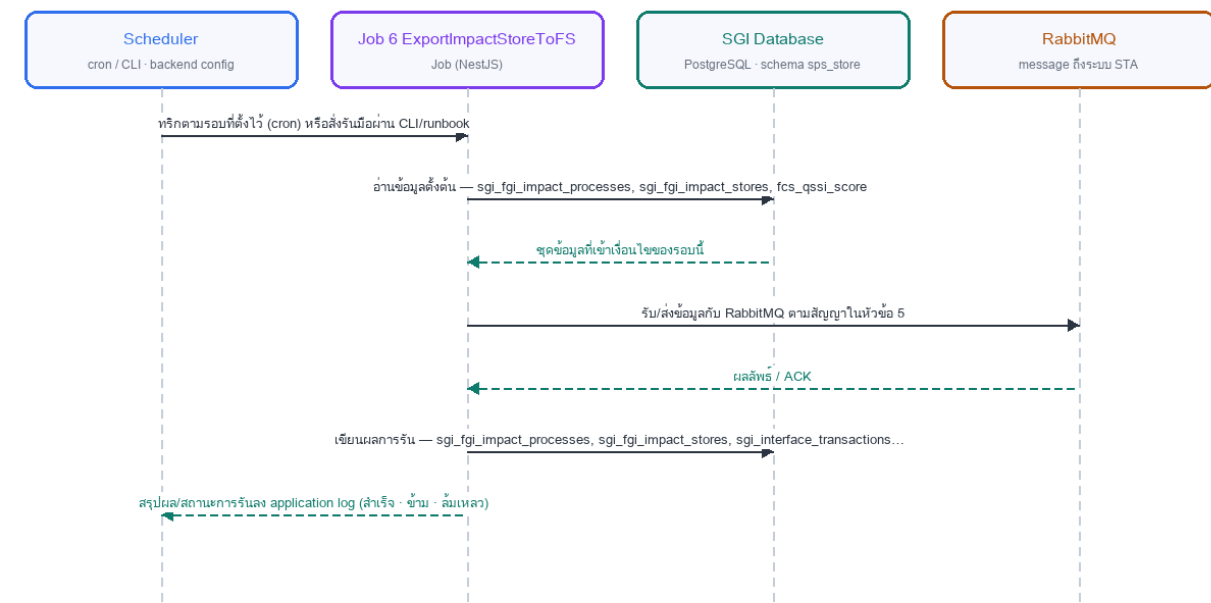


รูปที่ 1: Implementation flow reference: LLDD BE - Job 6 ExportImpactStoreToFS

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ

Sequence — LLDD BE - Job 6 ExportImpactStoreToFS



เส้นทึบ = การเรียก · เส้นประ = การตอบกลับ · ลูกปัดตัว = งานที่ทำภายใน service เอง

รูปที่ 2: Sequence diagram: LLDD BE - Job 6 ExportImpactStoreToFS

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 17 * * *	แก้ไขได้	ทุกวัน 17:00
dateStartInitToSTA	7	แก้ไขได้	วันของเดือนที่เริ่มปล่อยสถานะ I, C
numWaitPay	3	แก้ไขได้	จำนวนงวดรอจ่าย
หมวด QSSI ที่ตรวจ	8, 9, 12, 1, 10, 16	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ต้องครบทั้ง 6 หมวดจากงวด max เดียว ในกรอบ 3 เดือน
RabbitMQ exchange	sgi.interface (topic, durable)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	อ่านชื่อ exchange จาก backend config (SGI_MQ_EXCHANGE) ไม่ hardcode
Routing key	sta.compensation.result	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	STA เป็นเจ้าของ queue ที่ bind มาที่ routing key นี้
Message payload	JSON UTF-8 · dataName = sgi_impact_store (14 ฟิลด์ · สัญญาที่ ประกันรายได้ ตัวอย่าง Message RabbitMQ)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ฟิลด์ 3/5/6 คงเป็นวันที่ พ.ศ. ตามสัญญา 14 ฟิลด์เดิมของ STA — แปลงเฉพาะตอนประกอบ payload ห้ามให้ปนเข้า DB/API
Message properties	persistent (delivery_mode=2) · message_id = sgi_interface_transactions.id	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	message_id เป็น idempotency key ให้ STA กันรับซ้ำ
Secret reference	secret/sgi/mq/sta	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	user/password ของ broker จาก Secret Manager · เชื่อมด้วย AMQPS (TLS verify-full)

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	Approved/initial compensation data from FGI impact/new-store tables plus QSSI score lookup and FS export configuration.
Progress	query rows for FS, generate compensation interface payload, insert/update compensate records, upload/export, backup, notify.
Output	FS outbound data and FGI compensation tables synchronized; run summary includes exported counts and file/status.

5.90 Job 6 Execution Stages

query rows for FS, generate compensation interface payload, insert/update compensate records, upload/export, backup, notify.

Order	Service step	Repository	Output / failure contract
1	loadApprovedCompensations	statementExportRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
2	buildStatementPayload	statementExportRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
3	enqueueStatementOutbox	statementExportRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
4	purgeAcknowledgedTracking	statementExportRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

5.91 Job 6 Run Evidence

Evidence	Job-specific value	Acceptance
Input identity	Approved/initial compensation data from FGI impact/new-store tables plus QSSI score lookup and FS export configuration.	snapshot input file/business key/period in run record
Output identity	FS outbound data and FGI compensation tables synchronized; run summary includes exported counts and file/status.	reconcile input, success, reject and skipped counts
Dedup proof	UNIQUE(data_name,direction,business_key,period_key); publisher confirm เปลี่ยน outbox_status ของ transaction เดิมเป็น CONFIRMED ไม่ insert แถวใหม่	rerun fixture produces no duplicate target business key
Transaction proof	สร้าง payload/checksum ก่อน แล้ว insert outbox READY; dispatcher publish และเปลี่ยน SENT + outbox_status = PUBLISHED แยก transaction; เมื่อได้ publisher confirm จาก broker จึงเปลี่ยน outbox_status = CONFIRMED + status = COMPLETED แบบ compare-and-set บนแถวเดิม (มติ 2026-09-08 ข้อ 2.13 — ไม่มี ACK ระดับธุรกิจจาก STA)	injected failure leaves no partial committed state outside documented boundary
Security proof	RabbitMQ broker ใช้ secretRef=secret/sgi/mq/sta, เชื่อมด้วย AMQPS (TLS 1.2+ verify-full); exchange/routing key มาจาก config ไม่ใช่ค่าที่ผู้ใช้แก้ไขได้ (dataName = sgi_impact_store ตามสัญญาใน ประกันรายได้ ตัวอย่าง Message RabbitMQ); credential rotation ไม่ต้องแก้เอกสารหรือ job param	config/log/error contains no plaintext secret

5.92 Legacy Java Source Reference

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java	19-68	Legacy main entrypoint for exporting impact-store compensation to FS.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	119-180, 386-970	Query FS export data and insert/update impact/new-store compensation records.

Line ranges refer to the legacy Java implementation under /Users/bank_mac/gosoft/java/SBP/fcsJar. Use these ranges to preserve business behavior while implementing the target Node job.

5.93 Target Repository and SQL Contract

Contract	Target implementation
Repository	statementExportRepository
Idempotency / dedup	UNIQUE(data_name,direction,business_key,period_key); publisher confirm เปลี่ยน outbox_status ของ transaction เดิมเป็น CONFIRMED ไม่ insert แถวใหม่
Transaction boundary	สร้าง payload/checksum ก่อน แล้ว insert outbox READY; dispatcher publish และเปลี่ยน SENT + outbox_status = PUBLISHED แยก transaction; เมื่อได้ publisher confirm จาก broker จึงเปลี่ยน outbox_status = CONFIRMED + status = COMPLETED แบบ compare-and-set บนแถวเดิม (มติ 2026-09-08 ข้อ 2.13 — ไม่มี ACK ระดับธุรกิจจาก STA)
Security	RabbitMQ broker ใช้ secretRef=secret/sgi/mq/sta, เชื่อมด้วย AMQPS (TLS 1.2+ verify-full); exchange/routing key มาจาก config ไม่ใช่ค่าที่ผู้ใช้แก้ไขได้ (dataName = sgi_impact_store ตามสัญญาใน ประกันรายได้ ตัวอย่าง Message RabbitMQ); credential rotation ไม่ต้องแก้เอกสารหรือ job param

Input / candidate query

```
SELECT d.doc_no, d.impact_process_id, s.id AS sales_summary_id,
       d.total_compensation_amount, q.score
FROM   sgi_compensation_documents d
JOIN   sgi_fgi_impact_sales_summaries s ON s.impact_process_id = d.impact_process_id
LEFT JOIN fcs_qssi_score q ON q.store_id = d.impact_store_code AND q.month = d.impact_month
JOIN LATERAL (
  SELECT c.result_category
  FROM   sgi_consideration_logs c
  WHERE  c.doc_no = d.doc_no
  ORDER BY c.action_datetime DESC
  LIMIT 1
) latest_decision ON latest_decision.result_category = 'APPROVE'
WHERE  d.status_code = '99'
AND NOT EXISTS (
  SELECT 1 FROM sgi_interface_transactions i
  WHERE i.data_name = 'COMPENSATE_APPROVE_I' AND i.direction = 'OUT'
  AND i.doc_no = d.doc_no AND i.status IN ('READY','SENT'));
```

Write / upsert query

```
-- bind ตามลำดับ: $1=run_id · $2=doc_no · $3=impact_process_id · $4=sales_summary_id · $5=period_key · $6=file_name · $7=file_checksum ·
$8=purge_data_names · $9=purge_batch_size
INSERT INTO sgi_interface_transactions
  (run_id, data_name, direction, status, doc_no, impact_process_id, sales_summary_id,
   business_key, period_key, file_name, file_checksum, outbox_status, purge_after)
VALUES ($1 /* run_id */, 'COMPENSATE_APPROVE_I', 'OUT', 'READY', $2 /* doc_no */, $3 /* impact_process_id */,
       $4 /* sales_summary_id */, $2 /* doc_no */, $5 /* period_key */, $6 /* file_name */, $7 /* file_checksum */, 'READY',
       CURRENT_TIMESTAMP + INTERVAL '365 days')
ON CONFLICT (data_name, direction, business_key, period_key) DO NOTHING;

WITH purge_candidates AS (
  SELECT id
  FROM   sgi_interface_transactions
  WHERE  data_name = ANY($8 /* purge_data_names */)
  AND status = 'COMPLETED'
  AND purge_after < CURRENT_TIMESTAMP
  AND legal_hold = FALSE
  ORDER BY id
  LIMIT $9 /* purge_batch_size */
  FOR UPDATE SKIP LOCKED
)
DELETE FROM sgi_interface_transactions i
USING purge_candidates p
WHERE i.id = p.id
RETURNING i.id, i.data_name, i.business_key;
```

5.94 Target Node Implementation

โครงสร้างนี้ระบุ service/repository เฉพาะงานและต้อง implement ตาม SQL, transaction, idempotency และ security contract ด้านบน โดยทุกชั้นต้องคืน metrics สำหรับ reconcile และ run history

```
export async function runLddBeJob6Exportimpactstoretofs(ctx, services) {
  const run = await services.jobRuns.acquire({
    jobNo: "6", period: ctx.period, triggeredBy: ctx.triggeredBy
  });

  try {
    ctx = { ...ctx, runId: run.id, repository: services.statementExportRepository };
    const step1 = await services.loadApprovedCompensations(ctx, undefined);
    const step2 = await services.buildStatementPayload(ctx, step1);
    const step3 = await services.enqueueStatementOutbox(ctx, step2);
    const step4 = await services.purgeAcknowledgedTracking(ctx, step3);
    const result = step4;
    await services.jobRuns.finish(run.id, "SUCCESS", result.metrics);
    return { runId: run.id, status: "SUCCESS", ...result };
  } catch (error) {
    await services.jobRuns.finish(run.id, "FAILED", {
      errorCode: error.code ?? "JOB_FAILED",
      errorMessage: error.message
    });
    throw error;
  }
}
```

5.95 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-export-impact-store-to-fs` ใน `src/main.ts` ของ SBP/srm-sps-spsap-sop-sgi-batch โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรับ	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	<code>env JOB_NAME</code>	<code>env INPUT</code> (JSON string)	<code>JOB_NAME=sgi-export-impact-store-to-fs INPUT='{ "processDate": "2026-06-16" }'</code> npm run start
AWS Batch (ตารางเวลาจริง)	<code>process.argv[3]</code>	<code>process.argv[2]</code> (JSON string)	<code>node dist/main.js '{ "processDate": "2026-06-16" }' sgi-export-impact-store-to-fs</code>

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron/@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: `args[0] = yyyyMMdd` (วันประมวลผล) · ไม่ส่ง = วันปัจจุบัน · `parse` ไม่ได้ = log Error
PARAMETER is not format แล้วจบแบบ FAIL

ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างดี — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วทำอะไร (default)	Validation	ตรงกับ argument เดิม
<code>processDate</code>	string	วันนี้	YYYY-MM-DD (ค.ศ.) · <code>parse</code> ไม่ได้ = <code>INVALID_JOB_INPUT</code>	<code>args[0]</code> (yyyyMMdd)
<code>dryRun</code>	boolean	false	true = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
<code>limit</code>	number	null	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกากการ validate ที่ทุก job ต้องทำเหมือนกัน — `parse INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เจียน ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ลม

- `processDate` ไม่ได้เป็นแค่ label — มันเปลี่ยนผลลัพธ์ 2 ทาง: (1) dd ของมันตัดสินว่าจะสร้างชุด I (Initial) ให้ STA หรือไม่ (2) เดือนก่อนหน้าของมันคืองวดที่ใช้ตรวจ QSSI · ดูหัวข้อเงื่อนไขตัดสิน

5.96 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 6 มี 2 สวิตช์ที่เปลี่ยนผลลัพธ์ของทั้งรอบ และทั้งคู่ถูกตัดสิน ก่อน เปิด transaction — implement ผิดจะส่งข้อมูลผิดชุดให้ STA โดยไม่มี error ให้เห็น

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
รอบนี้จะสร้างชุด I (Initial) ให้ STA หรือไม่	<code>processDate</code> (argument · ดู 5.95) เทียบกับค่าคงที่ <code>dateStartInitToSTA</code>	dd ของ <code>processDate</code> $\geq 7 \rightarrow$ สร้างชุด Initial ของงวด (log Create Data) · $< 7 \rightarrow$ ไม่สร้าง (log No Create Data) แต่ยังส่งชุด A/N/S ตามปกติ	ค่านี้อยู่ใน <code>ApplicationResources.properties</code> ของระบบเดิม $\rightarrow$ ระบบใหม่ต้องเป็น env ไม่ใช่ค่าฝังในโค้ด

คำถามที่โค้ดต้องตอบ	ตัดสินใจ (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ข้อมูล QSSI ของงวดครบหรือยัง	fcs_qssi_score (ตารางของระบบ SBP เดิม · SGI อ่านอย่างเดียว) — year · month · category	งวดที่ตรวจ = เดือนก่อนหน้า processDate · ต้องมีข้อมูล ครบทุกหมวดใน 6 หมวด 8, 9, 12, 1, 10, 16 · หมวดใดหมวดหนึ่ง count = 0 → ไม่ครบทันที (ระบบเดิม break แล้วตั้ง countRow = 0)	ไม่ครบ → ชุดที่ส่งออกเปลี่ยนไป (ไม่ใช่ job fail) · □□ ระบบเดิม กลืน exception ของคิวรีนี้เป็น 0 ด้วย — ระบบใหม่ต้องแยก "ไม่ครบ" ออกจาก "คิวรีพัง" และ log ต่างกัน
แถวไหนถูกส่งออก และส่งด้วยสถานะอะไร	sgi_fgi_impact_compensat ions.compensate_status	ส่ง A (อนุมัติชดเชย) · N (เห็นควรไม่ชดเชย) · S (หยุดชดเชย) เสมอ · เพิ่ม I เมื่อสวิตช์ข้อ 1 และ 2 จริงพร้อมกัน · Z (ยอดเป็นศูนย์) แปลงเป็น S เฉพาะใน payload — ใน DB ยังเป็น Z	R (Reflow) ไม่ได้มาจาก job นี้ — เกิดจากปุ่มเปิดพิจารณาใหม่ POST /sgi/document/{docNo}/ac tions (ดู LLDD-BE-API-Docum ent-Workflow-Actions.pdf §5.1c)
ปิดรอบชดเชยหรือพักไว้	sgi_fgi_impact_processes .flag_action	ส่งผลชดเชยของงวดครบแล้ว → Y → N (ปิดรอบ) · ยังรอจ่ายอีก → Y → W (พัก)	flag_action เป็น input ของ จุดเข้า flow ชั้นที่ 1 ใน Job 8b — เขียนผิดที่นี้ทำให้เอกสารรอบหน้าเขา ผิดชั้น

ช่องว่างที่เคยค้างของหัวข้อนี้ — ปิดครบแล้ว (เก็บไว้เป็นประวัติ)

ทุกข้อปิดแล้ว — เก็บตารางไว้เพื่อให้ตามรอยได้ว่าเคยค้างอะไรและปิดด้วยอะไร

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G5 □ ปิดแล้ว 2026-09-02	compensate_status VARCHAR(5) เดิม ไม่มีทั้ง CHECK และคำอธิบายโดเมนใน DDL และค่า 'C' ไม่มีนิยาม	อ่าน ExportJdbc.insertFgiImpactStor eCompensate() (บรรทัด 395-460) แล้วพบนิยามของ 'C' — ดูตาราง "โดเมนของ compensate_status" ด้านล่าง · เติมโดเมนครบ + กฎ map ลง DDL แล้ว (CHECK IN ('I', 'C', 'A', 'N', 'S', 'Z'))

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว

ไม่ขึ้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
dateStartInitToSTA	7	ApplicationResources.properties ของระบบเดิม
categoryQssi	8, 9, 12, 1, 10, 16 (6 หมวด)	ApplicationResources.properties
numWaitPay	3 งวด	ApplicationResources.properties
compensate_status ที่ STA รับได้	I · A · N · S · R	ประกันรายได้ ตัวอย่าง Message RabbitMQ §2.2
ค่าที่มีใน DB แต่ไม่ส่งดิบ ๆ	Z และ C → แปลงเป็น S เฉพาะใน payload (ใน DB คงค่าเดิม)	ชั้นที่ 4 ของ Job 6 · ExportJdbc จัด C/S/Z เป็นกลุ่มเดียวกันในทุก filter ปลายทาง (บรรทัด 366 · 1440)

โดเมนของ `compensate\_status` ครอบคลุมค่า (สืบจาก `FgiConstant` + `ExportJdbc`)

ค่าเหล่านี้ไม่ได้อยู่ใน DDL เดิมเลย · สืบจากโค้ดจริงของระบบเดิมทั้งหมด — C เป็นค่าที่ระบบเดิมตั้งตอน insert ไม่ใช่ค่าที่คนเลือก
จึงต้องรองรับตั้งแต่ migration รอบแรก

ค่าใน DB	หมายความว่าอะไร	ส่งอะไรให้ STA	ที่มาในโค้ดเดิม
I	Iinitial — ข้อมูลตั้งต้นของงวด (ร้านยังเปิดและสัญญายังไม่จบ)	ส่ง I	FGI_COMPESATE_STATUS_I

ค่าใน DB	หมายความว่าอะไร	ส่งอะไรให้ STA	ที่มาในโค้ดเดิม
C	Closed/Cancelled — ตอน insert แถวเวลานั้นพบว่า ร้านปิดไปแล้ว (mas_store.close_date <= งวด) หรือ สัญญา SBP ถูกยกเลิกก่อน/ในงวดนั้นด้วยเหตุ cancel_type IN ('01', '02', '03', '04', '08') — ตั้งแทน I ตั้งแต่แรก	ส่ง S (STA ไม่รับค่า C) — กลุ่มเดียวกับ S/Z ในทุก filter ปลายทาง	ExportJdbc บรรทัด 404 · 414 · 452 (CASE ตอน insert) · 366 · 1440 (filter)
A	Approve — อนุมัติชัดเจน	ส่ง A	FGI_COMPESATE_STATUS_A
N	Not Approve — เห็นควรไม่ชัดเจน	ส่ง N	FGI_COMPESATE_STATUS_N
S	Stop — หยุดชัดเจนประกันรายได้	ส่ง S	FGI_COMPESATE_STATUS_S
Z	Zero — ยอดชัดเจนของงวดเป็นศูนย์	ส่ง S (แปลงเฉพาะใน payload · ใน DB ยังเป็น Z)	FgiConstant.FLAG_VERIFY_Z
R	Reflow — เปิดพิจารณาใหม่	เป็นค่าของ message เท่านั้น ไม่เคยลง DB — สร้างโดย POST / sgi/document/{docNo}/actions	สเปก STA §2.2 + LLDD-BE-API -Document-Workflow-Actions.pdf §5.1c

SQL ตรวจสอบความครบของ QSSI — **ต้องแยกที่ละหมวด ห้าม 'IN (...)' รวบรวมทีเดียว**

ระบบเดิมมาตรวจ ที่ละหมวด แล้ว break ทันทีที่เจอหมวดว่าง · เขียนเป็น category IN (8,9,12,1,10,16) แล้วนับรวมจะได้ผลต่างกัน (มีข้อมูลหมวดเดียวกันก็ผ่าน) ซึ่ง ผิด และจะส่งชุด Initial ผิดให้ STA

```
-- bind ตามลำดับ: $1=qssi_year · $2=qssi_month · $3=qssi_categories
-- ตรวจที่ละหมวด · หมวดใดได้ 0 1 ในชุดที่เห็นถือว่า "ไม่ครบ"
SELECT q.category, COUNT(1) AS count_rec
FROM fcs_qssi_score q -- schema sps_store · SGI อ่านอย่างเดียว
WHERE q.year = $1 /* qssi_year */ -- ปีของ "เดือนก่อนหน้า processDate"
AND q.month = $2 /* qssi_month */ -- เดือนก่อนหน้า processDate
AND q.category = ANY($3 /* qssi_categories */) -- {8, 9, 12, 1, 10, 16}
GROUP BY q.category;
-- ครบ ก็ต่อเมื่อ จำนวนหมวดที่คืนมา = 6 และทุกหมวด count_rec > 0
-- คิวรี throw = ต้อง log error แล้วหยุดรอบ ห้ามกลืนเป็น "ไม่ครบ" แบบระบบเดิม
```

5.97 เขียนข้อมูลรอบชัดเจน (รับเข้าโครง 2026-08-21 · gap F8 + F1)

Job 6 คือ job เดียวที่เขียนตารางรอบชัดเจนในระบบเดิม — ExportService.manageDBToFs() เรียก 5 คำสั่งต่อกันเป็นชุด ระบบใหม่ต้องทำครบเหมือนเดิม แต่เขียนลงตารางของ SGI

ลำดับใน manageDBToFs()	ระบบเดิม (Oracle)	ระบบใหม่ (SGI)	ใช้ทำอะไรต่อ
updateFgiImpactStoreOnProcess(INITDATE)	FGI_IMPACT_STORE_ON_PROCESS · LAST_COMPENSATE_SEQ_NO + 1 เมื่อ FLAG_ACTION='Y' และเพิ่งชัดเจนเดือนที่แล้ว	sgi_fgi_impact_processes.last_compensate_seq_no += 1	เคสต่อเนื่อง (SEQ_NO > 1)
insertFgiImpactStoreOnProcess()	แถวใหม่ · LAST_COMPENSATE_SEQ = MAX+1 · SEQ_NO = 1 · FLAG_ACTION='Y' · DATASOURCE	sgi_fgi_impact_processes แถวใหม่ (last_compensate_seq · last_compensate_seq_no=1 · flag_action · datasource)	เปิดเรื่องใหม่ (SEQ_NO = 1)
insertFgiImpactStoreCompensate(...)	FGI_IMPACT_STORE_COMPENSATE · COMPENSATE_FORECAST / COMPENSATE_ADJUST ต้องงวด	sgi_fgi_impact_compensations (forecast_amount / adjust_amount)	นับยอด 0 ติดกันกี่เดือน (กติกาดูเดือน 1-3 / เดือนที่ 4)
insertFgiNewStoreCompensate(...)	FGI_NEW_STORE_COMPENSATE	sgi_document_new_stores.compensation_amount / compensate_percent	ยอดต่อร้านเปิดใหม่
updateCompleteImpactStoreOnProcess / FlagYToW	FLAG_ACTION Y→N / Y→W	sgi_fgi_impact_processes.flag_action	ปิดรอบ / ส่งกลับรอบตรวจ

□□ ImportJdbc.insertImpactStoreOnProcess() / updateImpactStoreOnProcess() มี SQL ชุดเดียวกันอยู่ในไฟล์ Import แต่ตรวจทั้ง src แล้ว ไม่มี call site จริง — เป็นโค้ดตาย ให้อยู่ ExportJdbc เป็นต้นแบบเท่านั้น

5.98 Tracking Retention / Purge SQL

Purge ทำได้เฉพาะแถว COMPLETED ที่ครบ purge_after และไม่อยู่ใน legal hold; ต้องรันเป็น batch จำกัดจำนวนเพื่อไม่ lock ตารางยาว

```
-- bind ตามลำดับ: $1=sta_data_names · $2=batch_size
WITH purge_candidates AS (
  SELECT id
  FROM sgi_interface_transactions
  WHERE status = 'COMPLETED'
  AND purge_after < CURRENT_TIMESTAMP
  AND legal_hold = FALSE
  AND data_name = ANY($1 /* sta_data_names */)
  ORDER BY id
  LIMIT $2 /* batch_size */
  FOR UPDATE SKIP LOCKED
)
DELETE FROM sgi_interface_transactions i
USING purge_candidates p
WHERE i.id = p.id
RETURNING i.id, i.data_name, i.business_key;
```

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
รันตามตารางเวลา	CRON	scheduler → runner (job 6)	อ่าน cron/พารามิเตอร์จาก backend config
รันนอกรอบ (manual/rerun)	CLI	CLI/ops runbook → runner (job 6)	guard ไม่ให้รันซ้อนด้วย distributed lock
แก้พารามิเตอร์/เปิด-ปิด job	CONFIG	แก้ backend config แล้ว deploy	ไม่มี endpoint และไม่มีหน้าจอบทความ — หน้า Flow Batch Job เป็น reference อย่างเดียว (2026-08-06)
ตรวจสอบผลการรัน	LOG	application log (structured)	ไม่มีตาราง job_run_histories แล้ว · ไฟล์/ACK ดูที่ sgi_interface_transactions

7. API Contract

เอกสารฉบับนี้ไม่มี endpoint ของตัวเอง — เป็นสัญญา/งานภายในที่เอกสารอื่นเรียกใช้ (ดูขอบเขตใน 5.90 Endpoint Implementation Contract) · รายการ endpoint ทั้ง 28 เส้นของ SGI อยู่ที่ LLDD-API.pdf และ API design

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
sgi_fgi_impact_processes	R/W	หนึ่งใน 10 mutation (สถานะ process / last_compensation_amount)
sgi_fgi_impact_stores	R/W	สถานะค่าชุดเชย I/C/A/N/S/Z และข้อมูลร้าน/ผู้อนุมัติ/ค่าชุดเชยร้านใหม่
fcs_qssi_score	R	ตรวจสอบความครบคะแนน 6 หมวด — อ่านอย่างเดียว ระบบ SBP เดิมเป็นคณนำเข้า (คอลัมน์จริง: store_id · category · month · year · score)
sgi_fgi_impact_sales_summaries	R	growth_rate_diff / total_working_days ที่ใช้คัดชุดส่งออก (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)
sgi_compensation_documents	R	ผลพิจารณาของงวดที่ต้องส่งไป STA (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)

Table / Object	R/W	Usage
sgi_consideration_logs	R	ผลการพิจารณาล่าสุดของเอกสาร ใช้ประกอบ compensate_status (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)
sgi_interface_transactions	W	outbox + tracking COMPENSATE_INIT / APPROVE (I,N) · direction = OUT · READY → PUBLISHED → CONFIRMED · typed FK = impact_process_id

9. Skeleton Code (Batch Job 6)

9.1 ไฟล์ที่ต้องสร้าง (Job 6) — บน `src/modules/srm-sps-spsap-sop-sgi-batch`

โครงไฟล์ของ Job 6 (fgi.main.ExportImpactStoreToFS เดิม) วางได้ src/modules/sgi/ ตาม convention ที่ repo นี้ใช้อยู่จริง (ดูตัวอย่างที่ src/modules/performance/import-qssi.service.ts): service ถือ logic ทั้งหมด, inject DataSource/repository ผ่าน TypeORM, ยิง raw SQL ได้ตรง, และ ไม่มี controller เพราะ repo นี้ไม่เปิด HTTP

สิ่งที่ reuse ได้ทันที ไม่ต้องเขียนใหม่ — ต่างจากแผนเดิมที่ตั้งไว้บน store-backend ซึ่งต้องสร้าง runner ทั้งหมดเอง: src/main.ts (dispatcher + BATCH_START/BATCH_END + runId + log ลง integration_log อัปเดตในมด) · src/modules/rabbitMQ/rabbitmq.service.ts (publishMessage) · src/shared/services/s3.service.ts · StatementService.decodeThaiFileContent() (WINDOWS-874 auto-detect) · @gosoft-sbp/email-lib · @srm/glb-log

□□ สิ่งที่ repo นี้ยังไม่มี และเป็นงานตั้งต้นจริง: (1) ไม่มี @Cron เลย — ตารางเวลาต้องตั้งเป็น AWS Batch scheduled event (2) ไม่มี distributed lock (pg_try_advisory_lock) — การกันรันซ้อนพึ่ง AWS Batch queue ถ้างานไหนรับความเสี่ยงนี้ไม่ได้ต้องเพิ่มเอง (3) publishMessage เป็น fire-and-forget ไม่มี publisher confirm/outbox — งานที่ต้องการ transactional outbox + ACK (Job 6) ต้องสร้างกลไกเพิ่ม ไม่ใช่ reuse ตรง ๆ (4) ยังไม่มี entity/ตาราง sgi_* แมแต่ตัวเดียว

Path	หน้าที่
src/modules/sgi/job-6-export-impact-store-to-fs.service.ts	คลาส ExportImpactStoreToFsService — execute(input) เป็น entry point เดียว เรียงตาม flow ของ Job 6 ที่ละขั้น ครอบ transaction เอง และจบด้วย structured log สรุป metrics (แบบเดียวกับ ImportQssiService.execute(body) ที่มีอยู่)
src/modules/sgi/job-6-export-impact-store-to-fs.service.spec.ts	unit test ของ service — repo นี้วาง spec ไว้ข้างไฟล์จริงเสมอ (jest + npm run test:ci มี coverage/SonarQube)
src/modules/sgi/dto/job-6-export-impact-store-to-fs-input.dto.ts	DTO ของ INPUT (JSON) พร้อม class-validator ตามตารางในหัวข้อ 9.2 — parse ไม่ผ่านต้อง fail ก่อนแตะ DB
src/modules/sgi/sgi.module.ts	NestJS module ของกลุ่มงานประกกันรายได้ — ผูก service ทุกตัวของ SGI เข้ากับ TypeOrmModule (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ)
src/main.ts	เพิ่ม case 'sgi-job-6-export-impact-store-to-fs': ในสวิตช์เดิม → await import('./modules/sgi/job-6-export-impact-store-to-fs.service') แล้ว app.get(ExportImpactStoreToFsService).execute(input) (ไฟล์กลางของทุก job — เป็นจุด merge conflict ที่ต้องระวัง)
src/entities/sgi-*.entity.ts	entity ของตาราง sgi_* ที่หัวข้อ Reference DB Mapping อ้างถึง — ยังไม่มีใน repo เลยสักตัว ต้องสร้างใหม่ทั้งหมด
src/config/config.ts	เพิ่ม export const sgiJob6Config ตามแบบของไฟล์นี้ (โปรเจกต์ไม่ใช่ registerAs) — ค่าคงที่ทางธุรกิจของ Job 6

การลงทะเบียนใน `src/main.ts` (job `sgi-job-6-export-impact-store-to-fs`)

```
// src/main.ts — เพิ่มเคสในสวิตช์เดิม (เรียงต่อจาก job ของ SGI ตัวก่อนหน้า)
case 'sgi-job-6-export-impact-store-to-fs': {
  const { ExportImpactStoreToFsService } = await import('./modules/sgi/job-6-export-impact-store-to-fs.service');
  const job6ExportImpactStoreToFsService = app.get(ExportImpactStoreToFsService);
  await job6ExportImpactStoreToFsService.execute(input); // input = JSON ที่ parse จาก INPUT/argv[2] แล้ว
  break;
}
```

main.ts เรียก StatementService.logInterfest('sgi-job-6-export-impact-store-to-fs', input) ให้อยู่แล้วก่อนเข้าสวิตช์ → ไม่ต้องเขียน log ลง integration_log เองซ้ำ และ BATCH_END ที่ท้ายไฟล์จะสรุป batchSize +

durationMs ให้อัตโนมัติ หน้าของ service คือ throw เมื่อทำงานไม่สำเร็จเท่านั้น

9.2 Config Schema ของ Job 6 (backend config / env)

ตารางเวลาของ Job 6 คือ 0 17 * * * (ทุกวัน 17:00) — □□ ตัวจริงตั้งที่ AWS Batch scheduled event ไม่ใช่ในโค้ด (repo นี้ไม่มี @Cron เลย) ค่า SGI_JOB6_CRON เก็บไว้เป็นเอกสารประกอบ/ตรวจสอบเท่านั้น · SGI_JOB6_ENABLED=false ให้ execute() จบทันทีแบบ SUCCESS พร้อม log เหตุผล (กันกรณี AWS Batch ยังยิงเข้ามา)

```
// src/config/config.ts — เพิ่มสล็อตนี้ต่อท้าย (repo ใช้ export const ไม่ใช่ registerAs)
// convention จริงของ sop-sgi-batch คือ `export const` ใน `src/config/config.ts` (ไม่ใช่ registerAs · ดูของเดิมที่
// `AppConfigModule` แบบ @Global แล้วอ่าน process.env ตรง ๆ) — โปรดเจตน์นี้ **ไม่ได้ใช้ registerAs**
// แมแต่จุดเดียว จึงประกาศเป็นคลาสให้รีวิว/ทดสอบเหมือน config ตัวอื่น
import { Injectable } from '@nestjs/common';

// TODO: Job 6 ไม่มีตาราง job_configs และไม่มี Job Admin API แล้ว (ตัดสินใจ 2026-08-06)
// TODO: ค่าทุกตัวอ่านจาก env/config file ของ backend เท่านั้น — เปลี่ยนค่า = แก้ config แล้ว deploy
export interface Job6Config {
  /** เปิด/ปิด job รอบถัดไปโดยไม่ต้อง deploy โค้ด */
  enabled: boolean;
  /** ตารางเวลาของ job นี้ — บันทึกไว้เพื่ออ้างอิงเท่านั้น ตัวจริงตั้งที่ AWS Batch scheduled event */
  cron: string;
  /** กำหนดการรัน (Cron) — ทุกวัน 17:00 */
  cron: string;
  /** dateStartInitToSta — วันของเดือนที่เริ่มปล่อยสถานะ I, C */
  dateStartInitToSta: number;
  /** numWaitPay — จำนวนงวดรอจ่าย */
  numWaitPay: number;
  /** หมวด QSSI ที่ตรวจ — ต้องครบทั้ง 6 หมวดจากงวด max เดียว ในกรอบ 3 เดือน */
  qssi: string;
  /** RabbitMQ exchange — อ่านชื่อ exchange จาก backend config ('SGI_MQ_EXCHANGE') ไม่ hardcode */
  rabbitMqExchange: string;
  /** Routing key — STA เป็นเจ้าของ queue ที่ bind มาที่ routing key นี้ */
  routingKey: string;
  /** Message payload — ฟิลด์ 3/5/6 คงเป็นวันที่ พ.ศ. ตามสัญญา 14 ฟิลด์เดิมของ STA — แปลงเฉพาะตอนประกอบ payload ห้ามให้ปนเข้า DB/API */
  messagePayload: string;
  /** Message properties — message_id เป็น idempotency key ให้ STA กันรับซ้ำ */
  messageProperties: string;
  /** Secret reference — user/password ของ broker จาก Secret Manager · เชื่อมด้วย AMQPS (TLS verify-full) */
  secretReference: string;
  /** ผู้รับอีเมลเมื่อ job ล้มเหลว — เก็บเป็น string คั่น comma ให้ตรง signature ของ
    `EmailLibService.sendMail({ mailTo })` ที่รับ string ไม่ใช่ string[] */
  mailTo: string;
}

@Injectable()
export class SgiJob6Config implements Job6Config {
  // TODO: ยืนยันค่า default ทุกตัวกับ Ops ก่อนขึ้น production (ไม่มีหน้าจอกำหนดค่าแล้ว)
  enabled = (process.env.SGI_JOB6_ENABLED ?? 'true') === 'true';
  cron = process.env.SGI_JOB6_CRON ?? '0 17 * * *';
  cron = process.env.SGI_JOB6_CRON ?? '0 17 * * *'; // TODO: แก้ผ่าน env/config file แล้ว deploy
  dateStartInitToSta = Number(process.env.SGI_JOB6_DATE_START_INIT_TO_STA ?? 7); // TODO: แก้ผ่าน env/config file แล้ว deploy
  numWaitPay = Number(process.env.SGI_JOB6_NUM_WAIT_PAY ?? 3); // TODO: แก้ผ่าน env/config file แล้ว deploy
  qssi = process.env.SGI_JOB6_QSSI ?? '8, 9, 12, 1, 10, 16'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  rabbitMqExchange = process.env.SGI_JOB6_RABBIT_MQ_EXCHANGE ?? 'sgi.interface (topic, durable)'; // TODO: ค่าคงที่ทางธุรกิจ —
  // เปลี่ยนต้องผ่านการอนุมัติ
  routingKey = process.env.SGI_JOB6_ROUTING_KEY ?? 'sta.compensation.result'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  messagePayload = process.env.SGI_JOB6_MESSAGE_PAYLOAD ?? 'JSON UTF-8 · dataName = sgi_impact_store (14 ฟิลด์ · สัญญาที่
  ประกันรายได้ ตัวอย่าง Message RabbitMQ)'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  messageProperties = process.env.SGI_JOB6_MESSAGE_PROPERTIES ?? 'persistent (delivery_mode=2) · message_id =
  sgi_interface_transactions.id'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  secretReference = process.env.SGI_JOB6_SECRET_REFERENCE ?? 'secret/sgi/mq/sta'; // TODO: ค่าคงที่ทางธุรกิจ —
  // เปลี่ยนต้องผ่านการอนุมัติ
  mailTo = process.env.SGI_JOB6_MAIL_TO ?? ''; // TODO: ผู้รับอีเมลแจ้ง error คั่นด้วย comma (เดิม: storeretention เมื่อ publish สำเร็จ (เลิกส่ง
  mailToBPM — ไม่มีไฟล์ BPM แล้ว))
}

// TODO: เพิ่ม SgiJob6Config ใน providers/exports ของ AppConfigModule (@Global) เหมือน AppConfig
```

9.3 Job Class — `run(ctx)` ของ Job 6 ที่ละชั้นตามผัง

9.3.1 สัญญาของชั้นกลาง (`runner.ts`) + โครง service ของ Job 6

service อ้าง JobRunContext / JobRunResult / JobState / JobFailedError — ทั้งหมดนิยาม ครั้งเดียวใน src/modules/sgi/sgi-job.types.ts (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ) และ service ต้องมี method ครบตามตารางชั้นตอนด้านล่าง มิฉะนั้น execute(input) จะเรียก method ที่ไม่มีอยู่

```
// src/modules/sgi/sgi-job.types.ts — สัญญากลางของทุก job ของ SGI (ประกาศครั้งเดียว ใช้ร่วมทั้ง 10 ฉบับ)

export interface JobRunContext {
```

```

jobNo: string;
period: string; // YYYYMM ของงวดที่รัน
triggeredBy: string; // 'CRON' | userId ที่สั่งรันนอกรอบ
params?: Record<string, string>;
}

export interface JobRunResult {
  event: 'job.finish';
  jobNo: string;
  jobName: string;
  status: 'SUCCESS' | 'SKIPPED' | 'SKIPPED_LOCKED' | 'FAILED';
  period: string;
  output: string;
  read: number; written: number; skipped: number; rejected: number;
  durationMs: number;
}

/** counter + ค่าที่ทุกชั้นของ job ใช้ร่วมกัน (service เป็นผู้สร้างผ่าน createState) */
export interface JobState {
  period: string;
  read: number; written: number; skipped: number; rejected: number;
  // TODO: เพิ่ม field เฉพาะของ job นี้ (เช่น rows ที่อ่านมา, path ไฟล์ที่เขียน)
  [key: string]: unknown;
}

/** error ที่ทำให้ job จบเป็น FAILED และส่งอีเมลแจ้งผู้ดูแล */
export class JobFailedError extends Error {
  constructor(public readonly code: string, message: string) { super(message); }
}

/** ให้ออกจาก transaction เมื่อสาขา NO บอกให้ข้ามงวด/เรคคอร์ด — runner สรุปรเป็น SKIPPED ไม่ใช่ FAILED */
export class JobSkippedError extends Error {}

// ExportImpactStoreToFsService — method ที่ job class เรียก (1 method ต่อ 1 ชั้นในตารางด้านบน)
import { Inject, Injectable } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import type { JobRunContext, JobState } from '../runner';
export type { JobState };

@Injectable()
export class ExportImpactStoreToFsService {
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  createState(ctx: JobRunContext): JobState {
    return { period: ctx.period, read: 0, written: 0, skipped: 0, rejected: 0 };
  }

  // รัน 10 mutation ตามลำดับ บน sgi_fgi_impact_processes และ sgi_fgi_impact_stores
  async step02Validate(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step02Validate: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
  }

  // QSSI ครบ 6 หมวด? (งวด max เดียว ในกรอบ 3 เดือน)
  // เงื่อนไขจริง (จากหัวข้อเงื่อนไขตัดสินใจ): ข้อมูล QSSI ของงวดครบหรือยัง
  // ตัดสินจาก: fcs_qssi_score (ตารางของระบบ SBP เดิม · SGI อ่านอย่างเดียว) — year · month · category
  // ผ่านเมื่อ: งวดที่ตรวจ = เดือนก่อนหน้า processDate · ต้องมีข้อมูล ครบทุกหมวดใน 6 หมวด 8, 9, 12, 1, 10, 16 · หมวดใดหมวดหนึ่ง count = 0 →
  // ไม่ครบทันที (ระบบเดิม break แล้วตั้ง countRow = 0)
  // ไม่ผ่านแล้วทำอะไร: ไม่ครบ → ชุดที่ส่งออกไป (ไม่ใช่ job fail) · □ □ ระบบเดิม กลืน exception ของคิวรีนี้เป็น 0 ด้วย — ระบบใหม่ต้องแยก "ไม่ครบ"
  // ออกจาก "คิวรีพัง" และ log ต่างกัน
  async check03ResolvePeriod(state: JobState): Promise<boolean> {
    throw new Error('check03ResolvePeriod: ยังไม่ได้ implement — ห้าม deploy ทั้งที่ยังไม่เขียนเงื่อนไขจริง');
  }

  // สร้างชุดสถานะส่งออก: A, N, S (+ I, C เมื่อวันที่ ≥ 7 และ QSSI ครบ + Z ค้าง)
  async step04Parse(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step04Parse: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
  }

  // มีแถวส่งออก?
  // เงื่อนไขจริง: ดูตารางในหัวข้อ "เงื่อนไขตัดสินใจ (Decision Rules)" ของเอกสารฉบับนี้
  async check05Condition(state: JobState): Promise<boolean> {
    throw new Error('check05Condition: ยังไม่ได้ implement — ห้าม deploy ทั้งที่ยังไม่เขียนเงื่อนไขจริง');
  }

  // ประกอบ payload JSON UTF-8 14 บิต (วันที่ พ.ศ. ตามสัญญาเดิม)
  async step06Parse(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step06Parse: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
  }

  // insert outbox: I,C → COMPENSATE_INIT_I/N · A,N,S,Z → COMPENSATE_APPROVE_I/N (direction = OUT · status = READY)
  async step07Insert(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step07Insert: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
  }
}

```

```

// publish ไป RabbitMQ exchange sgi.interface (routing sta.compensation.result)
async step08Publish(state: JobState, manager?: EntityManager): Promise<void> {
  throw new Error('step08Publish: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
}

// ได้ publisher confirm?
// เงื่อนไขจริง: ดูตารางในหัวข้อ "เงื่อนไขตัดสินใจ (Decision Rules)" ของเอกสารฉบับนี้
async check09Publish(state: JobState): Promise<boolean> {
  throw new Error('check09Publish: ยังไม่ได้ implement — ห้าม deploy ทั้งที่ยังไม่เขียนเงื่อนไขจริง');
}

// update outbox: outbox_status READY → PUBLISHED → CONFIRMED (บันทึก sent_at)
async step10Publish(state: JobState, manager?: EntityManager): Promise<void> {
  throw new Error('step10Publish: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาก่อนหัวข้อเงื่อนไขตัดสินใจ');
}
}

```

9.3.2 `run(ctx)` ของ Job 6

ทุกขั้นใน `run()` ตรงกับ flowchart ของ Job 6 หนึ่งต่อหนึ่ง (decision และ error path รวมอยู่ด้วย) — method ที่ต้อง implement ใน service ตามตารางนี้

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
1	start	เริ่ม	<code>createState()</code>	-
2	process	รัน 10 mutation ตามลำดับบน <code>sgi_fgi_impact_processes</code> และ <code>sgi_fgi_impact_stores</code>	<code>step02Validate()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
3	decision	QSSI ครบ 6 หมวด? (งวด max เดียว ในกรอบ 3 เดือน)	<code>check03ResolvePeriod()</code>	[branch] ข้ามเส้นทาง INIT — สาย APPROVE ยังไปต่อ
4	process	สร้างชุดสถานะส่งออก: A, N, S (+ I, C เมื่อวันที่ ≥ 7 และ QSSI ครบ + Z ค่า)	<code>step04Parse()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
5	decision	มีแถวส่งออก?	<code>check05Condition()</code>	[end] จบการทำงาน
6	process	ประกอบ payload JSON UTF-8 14 ฟิลด์ (วันที่ พ.ศ. ตามสัญญาเดิม)	<code>step06Parse()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
7	process	insert outbox: I,C → COMPENSATE_INIT_I/N · A,N,S,Z → COMPENSA TE_APPROVE_I/N (direction = OUT · status = READY)	<code>step07Insert()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
8	io	publish ไป RabbitMQ exchange <code>sgi.interface</code> (routing <code>sta.compensation.result</code>)	<code>step08Publish()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
9	decision	ได้ publisher confirm?	<code>check09Publish()</code>	[err] คง outbox เป็น READY/FAILED_RETRY ให้ dispatcher ส่งซ้ำ — ไม่ rollback การ sync
10	process	update outbox: outbox_status READY → PUBLISHED → CONFIRMED (บันทึก sent_at)	<code>step10Publish()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
11	end	จบ	<code>summarize()</code>	-

```

// src/modules/sgi/job-6-export-impact-store-to-fs.service.ts — execute(input) เป็น entry point เดียว
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import { ExportImpactStoreToFsService, type JobState } from './job-6-export-impact-store-to-fs.service';
// 4 symbol นิยามใน src/modules/sgi/sgi-job.types.ts (โฟลเดอร์ของทุก job — ดูหัวข้อก่อนหน้า)
import { JobFailedError, JobSkippedError, JobRunContext, JobRunResult } from './runner';

```



```

@Injectable()
export class ExportImpactStoreToFsJob {
  static readonly jobNo = '6';
  private readonly logger = new Logger(ExportImpactStoreToFsJob.name);

  constructor(
    // TODO: repo นี้ตั้ง replication ใน typeorm.config.ts อยู่แล้ว — SELECT ไป slave, write ไป master อัตโนมัติ
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly service: ExportImpactStoreToFsService,
  ) {}

  async run(ctx: JobRunContext): Promise<JobRunResult> {
    const startedAt = Date.now();
    // TODO: state ถือ counter (read/written/skipped/rejected) และค่าจาก job6Config
    const state = this.service.createState(ctx);
    try {
      // ขั้นที่ 2: รัน 10 mutation ตามลำดับ บน sgi_fgi_impact_processes และ sgi_fgi_impact_stores · TODO: state sync ก่อน export — ตรวจครบทั้ง 10
      ขั้นตอน post-run
      await this.service.step02Validate(state);
      // ขั้นที่ 3 (decision): QSSI ครบ 6 หมวด? (งวด max เดียว ในกรอบ 3 เดือน) · TODO: หมวด 8, 9, 12, 1, 10, 16 จากคอลัมน์ category ของ
      fcs_qssi_score
      const ok03 = await this.service.check03ResolvePeriod(state);
      if (!ok03) { // NO → ข้ามเส้นทาง INIT — สาย APPROVE ยังไปต่อ
        // TODO: เส้น NO ของขั้นนี้เป็น branch ระดับ record — ผังไม่ใคร่บ่หายหรือไปต่อ
        // ถ้าเป็น 'ข้ามรายการ' -> state.skipped += 1; แล้ว continue ใน loop ของ record
        // ถ้าเป็น 'ตั้งค่าแล้วไปต่อ' -> เรียก service ตั้งค่าสถานะ แล้วเดินขั้นถัดไป (ห้าม return)
        // ถ้าเป็น 'คงสถานะเดิม/ไม่เปลี่ยน' -> หยุดเฉพาะ record นี้ ห้ามไหลไปขั้นถัดไป
      }
      // === transaction boundary === TODO: DB transaction คลุม 10 mutation + outbox (READY) เท่านั้น — publish อยู่นอก transaction แล้วค่อย
      update READY → PUBLISHED → CONFIRMED
      await this.dataSource.transaction(async (manager: EntityManager) => {
        // ขั้นที่ 4: สร้างชุดสถานะส่งออก: A, N, S (+ I, C เมื่อวันที่ ≥ 7 และ QSSI ครบ + Z ค้าง) · TODO: Z จะถูกแปลงเป็น S เฉพาะใน payload ที่ส่งออก —
        ใน DB ยังเป็น Z
        await this.service.step04Parse(state, manager);
        // ขั้นที่ 5 (decision): มีแถวส่งออก?
        const ok05 = await this.service.check05Condition(state);
        if (!ok05) { // NO → จบการทำงาน
          throw new JobSkippedError('NO branch'); // ใน transaction: โยนออกเพื่อ rollback
          // runner จับ JobSkippedError แล้วสรุปเป็น SKIPPED (ไม่ใช่ FAILED)
        }
        // ขั้นที่ 6: ประกอบ payload JSON UTF-8 14 필ด์ (วันที่ พ.ศ. ตามสัญญาเดิม) · TODO: 필ด์ผิด/แปลงวันที่ไม่ได้แม้แถวเดียว = ยกเลิกทั้งรอบ
        (คงพฤติกรรม mapData เดิม)
        await this.service.step06Parse(state, manager);
        // ขั้นที่ 7: insert outbox: I, C → COMPENSATE_INIT_I/N · A, N, S, Z → COMPENSATE_APPROVE_I/N (direction = OUT · status = READY) ·
        TODO: อยู่ใน transaction เดียวกับ 10 mutation — commit แล้วข้อมูลจะไม่หาย แม้ broker ล่ม
        await this.service.step07Insert(state, manager);
      });
      // ขั้นที่ 8: publish ไป RabbitMQ exchange sgi.interface (routing sta.compensation.result) · TODO: นอก DB transaction · persistent +
      publisher confirm + mandatory
      await this.service.step08Publish(state);
      // ขั้นที่ 9 (decision): ได้ publisher confirm? · TODO: เลี่ยง dual-write: การ sync สถานะกับการส่ง message แยก commit กัน · ส่งซ้ำได้เพราะ STA
      กันซ้ำด้วย message_id
      const ok09 = await this.service.check09Publish(state);
      if (!ok09) throw new JobFailedError('JOB6_STEP09', 'คง outbox เป็น READY/FAILED_RETRY ให้ dispatcher ส่งซ้ำ — ไม่ rollback การ sync');
      // ขั้นที่ 10: update outbox: outbox_status READY → PUBLISHED → CONFIRMED (บันทึก sent_at) · TODO: CONFIRMED ตั้งได้เฉพาะเมื่อได้
      publisher confirm จาก broker · Job 10 เฝ้าแถวที่ยังไม่ CONFIRMED ≥ 1 วัน
      await this.service.step10Publish(state);
      return this.summarize(state, 'SUCCESS', startedAt);
    } catch (error) {
      // TODO: error path ของ Job 6 — บั๊กจริง E20 ของโค้ดเดิม: SQL purge ต่อ data_name สองค่าเป็น string เดียว — tracking ไม่เคยถูกลบ
      // สสมโตขึ้นเรื่อย ๆ
      this.logger.error(JSON.stringify({ event: 'job.failed', jobNo: '6', period: ctx.period,
        triggeredBy: ctx.triggeredBy, durationMs: Date.now() - startedAt, error: (error as Error).message }));
      // TODO: แจ้งผู้ดูแลผ่าน JobFailureNotifier (หัวข้อ 9.6.1) — runner เป็นผู้เรียกให้
      throw error;
    }
  }

  private summarize(state: JobState, status: JobRunResult['status'], startedAt = Date.now()): JobRunResult {
    // TODO: structured log บรรทัดเดียวจบ — ไม่มีตาราง job_run_histories แล้ว (2026-08-06)
    const summary = {
      event: 'job.finish', jobNo: '6', jobName: 'ExportImpactStoreToFS', status,
      period: state.period, output: 'RabbitMQ message sgi_impact_store (exchange sgi.interface)',
      read: state.read, written: state.written, skipped: state.skipped,
      rejected: state.rejected, durationMs: Date.now() - startedAt,
    };
    this.logger.log(JSON.stringify(summary));
    return summary as JobRunResult;
  }
}

```

9.4 การกันรันซ้อนของ Job 6 (PostgreSQL advisory lock — **ของใหม่**)

Job 6 มีข้อควรระวังจาก legacy: บั๊กจริง E20 ของโค้ดเดิม: SQL purge ต่อ data_name สองค่าเป็น string เดียว — tracking ไม่เคยถูกลบ สะสมโตขึ้นเรื่อย ๆ — runner ล็อกด้วย pg_try_advisory_lock ก่อนเริ่มขั้นแรกเสมอ และรอบที่ล็อกไม่ได้ให้จบด้วยสถานะ SKIPPED_LOCKED (ไม่ใช่ FAILED)

```
// src/modules/sgi/sgi-job.lock.ts (ของใหม่ — repo นี้ยังไม่มีกลไกกันรันซ้อนใด ๆ)
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource } from 'typeorm';

// TODO: ห้ามใช้แถวสถานะ RUNNING ในตารางเป็นตัวแทน (ไม่มีตาราง job_run_histories แล้ว)
// ใช้ PostgreSQL advisory lock ระดับ session แทน — ปลดล็อกอัตโนมัติเมื่อ connection หลุด
export const SGI_JOB_LOCK_CLASS_ID = 861000; // namespace ของระบบ SGI
export const JOB_LOCK_KEYS: Record<string, number> = { '6': 60 /* TODO: เพิ่มให้ครบทุก job */ };

@Injectable()
export class BatchRunner {
  private readonly logger = new Logger(BatchRunner.name);
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  async runExclusive<T>(jobNo: string, fn: () => Promise<T>): Promise<T | { status: 'SKIPPED_LOCKED' }> {
    // TODO: ต้องใช้ QueryRunner (connection เดียวกับ master) — dataSource.query() ของโปรเจกต์นี้
    // route SQL ที่ขึ้นต้นด้วย SELECT ไป slave pool ทำให้ lock ไปตกที่ replica คนละ connection
    const runner = this.dataSource.createQueryRunner('master');
    await runner.connect();
    const objectId = JOB_LOCK_KEYS[jobNo];
    try {
      const [{ locked }] = await runner.query(
        'SELECT pg_try_advisory_lock($1, $2) AS locked',
        [SGI_JOB_LOCK_CLASS_ID, objectId],
      );
      if (!locked) {
        // TODO: รอบนี้ข้ามไปเลย ๆ ไม่ถือเป็น error และไม่ต้องส่งอีเมล
        this.logger.warn(JSON.stringify({ event: 'job.skipped.locked', jobNo }));
        return { status: 'SKIPPED_LOCKED' };
      }
      return await fn();
    } finally {
      // TODO: ปลด lock ทุกกรณี แล้วคืน connection เข้า pool
      await runner.query('SELECT pg_advisory_unlock($1, $2)', [SGI_JOB_LOCK_CLASS_ID, objectId]);
      await runner.release();
    }
  }
}
```

9.5 Repository / SQL หลักของ Job 6

repository ของ Job 6 ประกาศเป็น factory provider ({provide: 'EXPORT_IMPACT_STORE_TO_FS_REPOSITORY', useFactory: (ds) => ds.getRepository(Entity), inject: [DataSource]}) แล้วยิง raw SQL ตามแบบ module ธุรกิจอื่นของ store-backend (schema sps_store มาจาก search_path)

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
sgi_fgi_impact_processes	R/W	หนึ่งใน 10 mutation (สถานะ process / last_compensation_amount)	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_fgi_impact_stores	R/W	สถานะค่าชดเชย I/C/A/N/S/Z และข้อมูลร้าน/ผู้อนุมัติ/ค่าชดเชยร้านใหม่	เขียน SQL ตรงผ่าน DATA_SOURCE
fcs_qssi_score	R	ตรวจสอบความครบคะแนน 6 หมวด — อ่านอย่างเดียว ระบบ SBP เดิมเป็นคนนำเขา (คอลัมน์จริง: store_id · category · month · year · score)	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_fgi_impact_sales_summaries	R	growth_rate_diff / total_working_days ที่ใช้ตัดชุดส่งออก (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_compensation_documents	R	ผลพิจารณาของงวดที่ต้องส่งไป STA (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)	เขียน SQL ตรงผ่าน DATA_SOURCE

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
sgi_consideration_logs	R	ผลการพิจารณาล่าสุดของเอกสาร ใช้ประกอบ compensate_status (เพิ่ม 2026-09-02 — SQL แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_interface_transactions	W	outbox + tracking COMPENSATE_INIT / APPROVE (I,N) · direction = OUT · READY → PUBLISHED → CONFIRMED · typed FK = impact_process_id	เขียน SQL ตรงผ่าน DATA_SOURCE

```
-- Job 6 ExportImpactStoreToFS — query หลักที่ต้อง implement
-- TODO: ทุก statement รันผ่าน DATA_SOURCE (SELECT ไป slave, write ไป master) และ
-- write ทั้งหมดต้องอยู่ใน transaction เดียวกันที่ระบุใน 9.3

-- [R/W] sgi_fgi_impact_processes : หนึ่งใน 10 mutation (สถานะ process / last_compensation_amount)
-- อ่าน candidate แบบล็อกแถว กันรอบอื่น/pod อื่นแย่งอัปเดตแถวเดียวกัน
SELECT id, action_status, created_at, datasource, end_compensate_month, end_compensate_year, flag_action, impact_month -- ดัดคอลัมน์ที่
job นี้ไม่ได้ไขออก (ทั้งตารางมี 19 คอลัมน์)
FROM sgi_fgi_impact_processes
WHERE impact_year = $1 AND impact_month = $2 -- คอลัมน์จริงของตารางนี้ตาม DDL
FOR UPDATE SKIP LOCKED;

UPDATE sgi_fgi_impact_processes
SET /* TODO: คอลัมน์สถานะ/ผลคำนวณที่ job นี้เขียน */
    updated_at = NOW(), updated_by = 'JOB6'
WHERE /* id ที่ล็อกไว้จาก SELECT ... FOR UPDATE ข้างบน */ id = ANY($1);

-- [R/W] sgi_fgi_impact_stores : สถานะค่าชดเชย I/C/A/N/S/Z และข้อมูลร้าน/ผู้อนุมัติ/ค่าชดเชยร้านใหม่
-- อ่าน candidate แบบล็อกแถว กันรอบอื่น/pod อื่นแย่งอัปเดตแถวเดียวกัน
SELECT id, adjust_compensate_percent, adjust_compensation_amount, created_at, created_by, distance_km, forecast_compensate_percent,
forecast_compensation_amount -- ดัดคอลัมน์ที่ job นี้ไม่ได้ไขออก (ทั้งตารางมี 16 คอลัมน์)
FROM sgi_fgi_impact_stores
WHERE impact_month = $1 -- คอลัมน์จริงของตารางนี้ตาม DDL
FOR UPDATE SKIP LOCKED;

UPDATE sgi_fgi_impact_stores
SET /* TODO: คอลัมน์สถานะ/ผลคำนวณที่ job นี้เขียน */
    updated_at = NOW(), updated_by = 'JOB6'
WHERE /* id ที่ล็อกไว้จาก SELECT ... FOR UPDATE ข้างบน */ id = ANY($1);

-- [R] fcs_qssi_score : ตรวจสอบความครบคะแนน 6 หมวด — อ่านอย่างเดียว ระบบ SBP เดิมเป็นคณนำเข้า (คอลัมน์จริง: store_id · category · month · year ·
score)
-- คอลัมน์มาจาก DDL จริงของตารางนี้ (ห้าม SELECT *) · ตรวจสอบว่ามี index รองรับ WHERE ก่อนขึ้น prod
SELECT id, store_id, category, month, year, score, create_date -- คอลัมน์จริงจาก db schema sps_store (ทั้งตารางมี 7 คอลัมน์) · ดัดที่ job
นี้ไม่ได้ไขออก
FROM fcs_qssi_score
WHERE category = $1 AND score_period = $2
-- □ □ ต้องวนตรวจ **ทีละหมวด** ตาม categoryQssi = 8,9,12,1,10,16
-- ห้ามใช้ category IN (...) รวมเดียว (ดูหัวข้อ SQL ตรวจสอบความครบของ QSSI)
ORDER BY id -- ตารางระบบเดิมไม่มี PK ที่ประกาศไว้ · ใช้คอลัมน์นี้ให้ลำดับคงที่
LIMIT $3 OFFSET $4; -- อ่านเป็น chunk กัน memory บวม

-- [R] sgi_fgi_impact_sales_summaries : growth_rate_diff / total_working_days ที่ใช้คัดชุดส่งออก (เพิ่ม 2026-09-02 — SQL
แต่อยู่แล้วแต่ไม่ได้ประกาศไว้)
-- คอลัมน์มาจาก DDL จริงของตารางนี้ (ห้าม SELECT *) · ตรวจสอบว่ามี index รองรับ WHERE ก่อนขึ้น prod
SELECT id, growth_rate_after, growth_rate_before, growth_rate_diff, impact_process_id, sales_status, total_working_days, updated_at,
updated_by -- ดัดคอลัมน์ที่ job นี้ไม่ได้ไขออก (ทั้งตารางมี 9 คอลัมน์)
FROM sgi_fgi_impact_sales_summaries
WHERE impact_process_id = $1 -- คีย์ที่ job นี้ใช้คัดแถว (ตารางนี้ไม่มีคอลัมน์จริงของตัวเอง)
ORDER BY id -- PK ทำให้ลำดับคงที่ระหว่างแบ่งหน้า
LIMIT $2 OFFSET $3; -- อ่านเป็น chunk กัน memory บวม
```

9.6 การแจ้งเตือนและการรันซ้ำของ Job 6

9.6.1 อีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว

ใช้ EmailLibService จาก @gosoft-sbp/email-lib ตัวเดียวกับที่ระบบเดิมใช้ (inform-evaluate / external-audit / statement PTT) — ไม่สร้างกลไกส่งเมลใหม่

```
// src/modules/sgi/sgi-job-failure.notifier.ts (ใช้ @gosoft-sbp/email-lib ที่ repo มีอยู่แล้ว)
import { Injectable, Logger } from '@nestjs/common';
// ชื่อ method ของ lib ที่ repo นี้เรียกจริงคือ 'sendMail' (ไม่ใช่ sendEmail) และ
// 'mailTo' / 'mailCc' เป็น **string** คั่นด้วย comma — ดู evaluation-process.service.ts,
// external-audit.service.ts, statement.service.ts, inform-evaluate.service.ts, performance.service.ts
import { EmailLibService } from '@gosoft-sbp/email-lib';
import type { JobRunContext } from './runner';
```

```

@Injectable()
export class JobFailureNotifier {
  private readonly logger = new Logger(JobFailureNotifier.name);
  // TODO: ใช้ lib อื่นๆของระบบเดิม — template อยู่ในตาราง email_template และ log ลง email_sent อัตโนมัติ
  // (ตั้งชื่อ property ว่า mailService ตาม call site เดิมของ sop-sgi-batch)
  constructor(private readonly mailService: EmailLibService) {}

  async notifyFailure(jobNo: string, ctx: JobRunContext, error: Error): Promise<void> {
    // TODO: ผู้รับของ Job 6 เดิมคือ storeretention เมื่อ publish สำเร็จ (เลิกส่ง mailToBPM — ไม่มีไฟล์ BPM แล้ว) — ย้ายมาเป็น env
    SGI_JOB6_MAIL_TO
    const recipients = (process.env.SGI_JOB6_MAIL_TO ?? '').split(',').map(s => s.trim()).filter(Boolean);
    if (!recipients.length) {
      this.logger.warn(JSON.stringify({ event: 'job.mail.skipped', jobNo, reason: 'NO_RECIPIENT' }));
      return;
    }
    try {
      await this.mailService.sendMail({
        // TODO: emailId = id ของ template EM-07 (แจ้ง error batch) ในตาราง email_template
        emailId: Number(process.env.SGI_JOB_FAIL_EMAIL_TEMPLATE_ID),
        mailTo: recipients.join(','), // signature รับ string ไม่ใช่ string[]
        mailCc: '',
        param: {
          jobNo, jobName: 'ExportImpactStoreToFS',
          jobTitle: 'ซิงค์สถานะ + ส่งคำขอตกลง STA',
          period: ctx.period, triggeredBy: ctx.triggeredBy,
          output: 'RabbitMQ message sgi_impact_store (exchange sgi.interface)',
          errorMessage: error.message,
          rerunNote: 'transaction ป้องกันตามปกติ แต่ต้อง reconcile 10 mutation · ส่งซ้ำปลอดภัยเพราะ message_id = sgi_interface_transactions.id ให้ STA กันซ้ำได้',
        },
      });
    } catch (mailError) {
      // TODO: ส่งเมลไม่สำเร็จห้ามกลบ error เดิมของ job — log แล้วปล่อยผ่าน
      this.logger.error(JSON.stringify({ event: 'job.mail.failed', jobNo, error: (mailError as Error).message }));
    }
  }
}

```

9.6.2 Checklist การ rerun

- กติกา rerun ของ Job 6: transaction ป้องกันตามปกติ แต่ต้อง reconcile 10 mutation · ส่งซ้ำปลอดภัยเพราะ message_id = sgi_interface_transactions.id ให้ STA กันซ้ำได้
- ขอบเขต transaction ที่ต้องรักษาเมื่อรันซ้ำ: DB transaction คลุม 10 mutation + outbox (READY) เท่านั้น — publish อยู่นอก transaction แล้วค่อย update READY → PUBLISHED → CONFIRMED
- ความเสี่ยงที่ต้องตรวจสอบ/หลังรันซ้ำ: บั๊กจริง E20 ของโค้ดเดิม: SQL purge ต่อ data_name สองค่าเป็น string เดียว — tracking ไม่เคยถูกลบ สะสมโตขึ้นเรื่อย ๆ
- ตรวจสอบว่ารอบก่อนหน้าไม่ได้ค้าง lock อยู่ (SELECT * FROM pg_locks WHERE locktype = 'advisory') ก่อนสั่งรันนอกรอบ
- สั่งรันนอกรอบผ่าน CLI/runbook เท่านั้น (ไม่มีหน้าจอลงและไม่มี Job Admin API): node dist/batch/cli.js --job=6 --period=<YYYYMM>
- หลังรันซ้ำ ตรวจสอบ output RabbitMQ message sgi_impact_store (exchange sgi.interface) และ log บรรทัด job.finish ว่า read/written/skipped/rejected ตรงกับที่คาด
- ถ้ารอบก่อนล้มเหลวกลางทาง ตรวจสอบ sgi_interface_transactions ของงวดนั้นว่ามีแถวค้างสถานะ READY/PENDING หรือไม่ ก่อนสั่งรันใหม่

10. Processing Flow

Step	Description
1	เริ่ม
2	รัน 10 mutation ตามลำดับ บน sgi_fgi_impact_processes และ sgi_fgi_impact_stores (state sync ก่อน export — ตรวจสอบครบทั้ง 10 ขั้นตอน post-run)
3	QSSI ครบ 6 หมวด? (งวด max เดียว ในกรอบ 3 เดือน) No: ข้ามเส้นทาง INIT — สาย APPROVE ยังไปต่อ (หมวด 8, 9, 12, 1, 10, 16 จากคอลัมน์ category ของ fcs_qssi_score)

Step	Description
4	สร้างชุดสถานะส่งออก: A, N, S (+ I, C เมื่อวันที่ ≥ 7 และ QSSI ครบ + Z ค้าง) (Z จะถูกแปลงเป็น S เฉพาะใน payload ที่ส่งออก — ใน DB ยังเป็น Z)
5	มีแถวส่งออก? No: จบการทำงาน
6	ประกอบ payload JSON UTF-8 14 필ด์ (วันที่ พ.ศ. ตามสัญญาเดิม) (ฟิลด์ผิด/แปลงวันที่ไม่ได้แม้แถวเดียว = ยกเลิกทั้งรอบ (คงพฤติกรรม mapData เดิม))
7	insert outbox: I,C → COMPENSATE_INIT_I/N · A,N,S,Z → COMPENSATE_APPROVE_I/N (direction = OUT · status = READY) (อยู่ใน transaction เดียวกับ 10 mutation — commit แล้วข้อมูลจะไม่หาย แม้ broker ล่ม)
8	publish ไป RabbitMQ exchange sgi.interface (routing sta.compensation.result) (นอก DB transaction · persistent + publisher confirm + mandatory)
9	ได้ publisher confirm? No: คง outbox เป็น READY/FAILED_RETRY ให้ dispatcher ส่งซ้ำ — ไม่ rollback การ sync (เสี่ยง dual-write: การ sync สู่สถานะกับการส่ง message แยก commit กัน · ส่งซ้ำได้เพราะ STA กันซ้ำด้วย message_id)
10	update outbox: outbox_status READY → PUBLISHED → CONFIRMED (บันทึก sent_at) (CONFIRMED ตั้งได้เฉพาะเมื่อได้ publisher confirm จาก broker · Job 10 ฝ่าแถวที่ยังไม่ CONFIRMED ≥ 1 วัน)
11	จบ

11. Acceptance Criteria

- พารามิเตอร์รับผ่าน INPUT (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ @Cron ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + runId — src/main.ts ทำให้แล้ว) และบันทึกลง integration_log อัปเดตโน้ต · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช้งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

12. Developer Test Checklist

No	Test
1	รันตามตารางเวลาแล้วผลถูกต้องบน fixture
2	รันนอกรอบผ่าน CLI ได้ผลเดียวกับ cron
3	สั่งรันซ้อนขณะกำลังรัน → runner ปฏิเสธ (lock ทำงาน)
4	แก้ config แล้ว deploy → รอบถัดไปใช้ค่าใหม่
5	job throw error → EM-07 ออก และ log มีบรรทัด error
6	ตรวจผลกระทบตารางตาม R/W mapping reference

13. Unit Test Scope

8 ชั่วโมง (30% ของ implementation 24 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
business rule	logic	พารามิเตอร์รับผ่าน INPUT (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ @Cron ในโค้ด
business rule	logic	การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนทั้ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
business rule	logic	ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + runId — src/main.ts ทำให้อ่านง่าย) และบันทึก integration_log อัปเดตโน้ต · error ต้องส่ง EM-07
business rule	logic	DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
business rule	logic	รองรับ rerun rule และ risk note ตาม runbook
sgi_fgi_impact_processes, sgi_fgi_impact_stores, sgi_interface_transactions	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
runner	idempotency	รันซ้ำด้วย fixture เดิมต้องไม่เกิดแถวซ้ำ (ON CONFLICT / business unique key ทำงาน)
runner	lock	เรียกซ้อนขณะกำลังรัน ต้องถูกปฏิเสธด้วย advisory lock

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportImpactStoreToFS.java — lines 19-30

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java
Original lines	19-30
Responsibility	Legacy main entrypoint for exporting impact-store compensation to FS.

```
public class ExportImpactStoreToFS {
    private static ApplicationContext context = new FileSystemXmlApplicationContext(
        "config.xml");
    private static ExportController exportController = (ExportController) context.getBean("exportController");
    private static final FgiConfig config = (FgiConfig) context.getBean("fgiConfig");

    public static void main(String[] args){
        String parameter = null;
        boolean isParameter=false, isCompleted=false, isCreateData=false;
        Calendar StartDateTime = Calendar.getInstance(Locale.US);
        final EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
        informBean.setSystemCode(FgiConstant.SYSTEM_CODE);
    }
}
```

J1. ExportImpactStoreToFS.java — lines 57-68

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java
Original lines	57-68
Responsibility	Legacy main entrypoint for exporting impact-store compensation to FS.

```

    }else{
        informBean.setStatus(FgiConstant.JOB_STATUS_SUCCESS);
    }
    SendMailUtil.sendMailToAdmin(informBean, config, true);
}

// String endTime = DateUtils.getCurrentDate(DateUtils.LOCALE_EN,FgiConstant.DATE_FORMAT_DDMMYYYY_TIME)+"";
LogUtils.info(ExportImpactStoreToFS.class,"End => export store compensate to STA");
}
}

```

J2. ExportJdbc.java — lines 119-130

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	119-130
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```

public List<Map<String, String>> queryFmlImpactStoreToFS(boolean isQssi, boolean isCreateInitToSTA, final String NUM_WAIT_PAY, final String INITDATE){
    StringBuffer sql = new StringBuffer();
    String compensate_status = ""+FgiConstant.FGI_COMPESATE_STATUS_A+"", ""+FgiConstant.FGI_COMPESATE_STATUS_N+"",
    ""+FgiConstant.FGI_COMPESATE_STATUS_S+"";
    if(isCreateInitToSTA&&isQssi){
        compensate_status=""+FgiConstant.FGI_COMPESATE_STATUS_I+"", ""+FgiConstant.FGI_COMPESATE_STATUS_C+"",
        ""+FgiConstant.FGI_COMPESATE_STATUS_A+"", ""+FgiConstant.FGI_COMPESATE_STATUS_N+"", ""+FgiConstant.FGI_COMPESATE_STATUS_S+"";
    }

    sql.append(" select fisi.storecode_i, fnsi.storecode_n, fnsi.opendate_n, ");
    sql.append(" count(fnsc.storecode_n) over(partition by fnsc.impact_process_id, fnsc.compensate_month, fnsc.compensate_year) as count_storecode_n, ");
    sql.append(" fisc.compensate_month, fisc.compensate_year, ");
    sql.append(" fisc.stmt_month, fisc.stmt_year, ");
    sql.append(" decode(fisc.compensate_status, ""+FgiConstant.FLAG_VERIFY_Z+"", ""+FgiConstant.FGI_COMPESATE_STATUS_S+"", fisc.compensate_status) as compensate_status, ");
}

```

J2. ExportJdbc.java — lines 169-180

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	169-180
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```

sql.append(" left join fcs_qssi_score qs1 on qs1.store_id = qs.store_id ");
sql.append(" and to_date(qs1.year || qs1.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs1.category = 8 ");
sql.append(" left join fcs_qssi_score qs2 on qs2.store_id = qs.store_id ");
sql.append(" and to_date(qs2.year || qs2.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs2.category = 9 ");
sql.append(" left join fcs_qssi_score qs3 on qs3.store_id = qs.store_id ");
sql.append(" and to_date(qs3.year || qs3.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs3.category = 12 ");
sql.append(" left join fcs_qssi_score qs4 on qs4.store_id = qs.store_id ");
sql.append(" and to_date(qs4.year || qs4.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs4.category = 1 ");

```


J2. ExportJdbc.java — lines 386-397

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	386-397
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```
public int insertFgiImpactStoreCompensate(final List<ImpactStore> impactStoreList, final String flag){
    int result=0;
    String sqlAppend = "";
    if(FgiConstant.FRANCHISE_STATEMENT.equalsIgnoreCase(flag)){
        sqlAppend = " and op.storecode_i = ? "
        +" and op.last_compensate_month = ? "
        +" and op.last_compensate_year = ? ";
    }
    StringBuilder sql = new StringBuilder();
    sql.append(" insert into fgi_impact_store_compensate ( ");
    sql.append(" compensate_i_id, impact_process_id, storecode_i, compensate_seq, compensate_seq_no, ");
    sql.append(" compensate_month, compensate_year, compensate_forecast, compensate_adjust, compensate_status, ");
```

J2. ExportJdbc.java — lines 959-970

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	959-970
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```

sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.opendate_i, ms.open_date) as opendate_i, ");
sql.append(" ms.close_date as closedate_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.zone_i, ms.region) as zone_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.branchtype_i, ms.status_type) as branchtype_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.juristic_id_i, j.juristic_id) as juristic_id_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.juristic_name_i, j.juristic_name) as juristic_name_i, ");
sql.append(" fz.first_name as franchisee_fname_i, fz.last_name as franchisee_lname_i, ");
sql.append(" case when (op.datasource = '"+FgiConstant.FRANCHISE_STATEMENT+"' ) then ");
sql.append(" '04' ");
sql.append(" when (fiss.flag_verify = '"+FgiConstant.FLAG_VERIFY_ACCEPT+"' and (fiss.total_working_day != '"+FgiConstant.TOTAL_INTERVAL_DAY+"' or
fiss.growth_rate_diff is null)) then ");
sql.append(" '03' ");
sql.append(" when ( ");

```